숨결톡톡 | RESONANCE ANALYSIS

로그 스케일 적용 및 공진 후보 탐색 알고리즘

수식 검증 · 주파수 보정 · Arduino IDE 전체 소스코드

이번 개정에서 바뀌는 것

원래 그래프	X=Frequency(Hz), Y=Acceleration(m/s^2) 유지
로그 계산	가속도 dB와 속도 보정 dB를 따로 출력
후보 판정	속도 환산값의 내부 피크만 예비 후보로 보고
구동 방식	기존 구동부는 변경하지 않음; 이 소스에는 구동부 미포함

핵심: 로그 변환 자체는 공진 위치를 바꾸지 않음.

속도 환산과 내부 피크 판정을 결합하되, 입력 가진력 미측정 한계는 남음.

작성일 2026.11.03 | 센서 BNO085 | Arduino IDE (ESP32)

DESIGN NOTE

01. 로그 스케일의 의미

로그 표시와 물리적 주파수 보정을 반드시 구분한다.

A. 로그만 적용하면

현재 가속도 RMS를 그대로 두고 Y축만 dB로 바꾸는 방법이다. 기준 가속도를 1 m/s^2 로 명시한다.

$$L_a(f) = 20\log_{10}\left(\frac{A_{\mathrm{RMS}}(f)}{A_{\mathrm{ref}}}\right)$$

로그 함수는 양수 구간에서 단조 증가한다. 따라서 원래 가속도가 45 Hz에서 최대라면, 로그 가속도도 45 Hz에서 최대다.

B. 속도 환산은 별도의 물리 계산

$$A(f) = (2\pi f)^2 X(f)$$

정현파 변위의 두 번째 시간미분이 가속도이므로, 같은 변위에서는 주파수의 제곱이 가속도 진폭에 곱해진다.

$$V_{\mathrm{RMS}}(f) = \frac{A_{\mathrm{RMS}}(f)}{2\pi f}$$

구동 주파수 성분의 가속도 RMS를 각주파수로 나누면 속도 RMS가 된다. 기존 센서를 유지할 수 있는 부분적 보완이다.

> 주의: 가속도에서 속도로 환산해도 입력 가진력의 주파수 의존성은 제거되지 않는다. 따라서 자동 판정 문구는 반드시 예비 공진 후보로 제한한다.

02. 로그 보정식과 기준값

30 Hz는 표시용 기준 주파수이며 임상·기계적 정답이 아니다.

가속도 레벨 vs 속도 레벨

$$L_v(f) = 20\log_{10}\left(\frac{V_{\mathrm{RMS}}(f)}{V_{\mathrm{ref}}}\right)$$

여기서 Vref = 0.001 m/s(1 mm/s)이다. 속도 dB는 가속도 dB와 다른 참조 단위를 사용하므로 절대 dB 수치를 서로 비교하면 안 된다.

기준 주파수를 이용한 보정 레벨

$$L_{\mathrm{corr}}(f) = L_a(f) - 20\log_{10}\left(\frac{f}{f_{\mathrm{ref}}}\right)$$

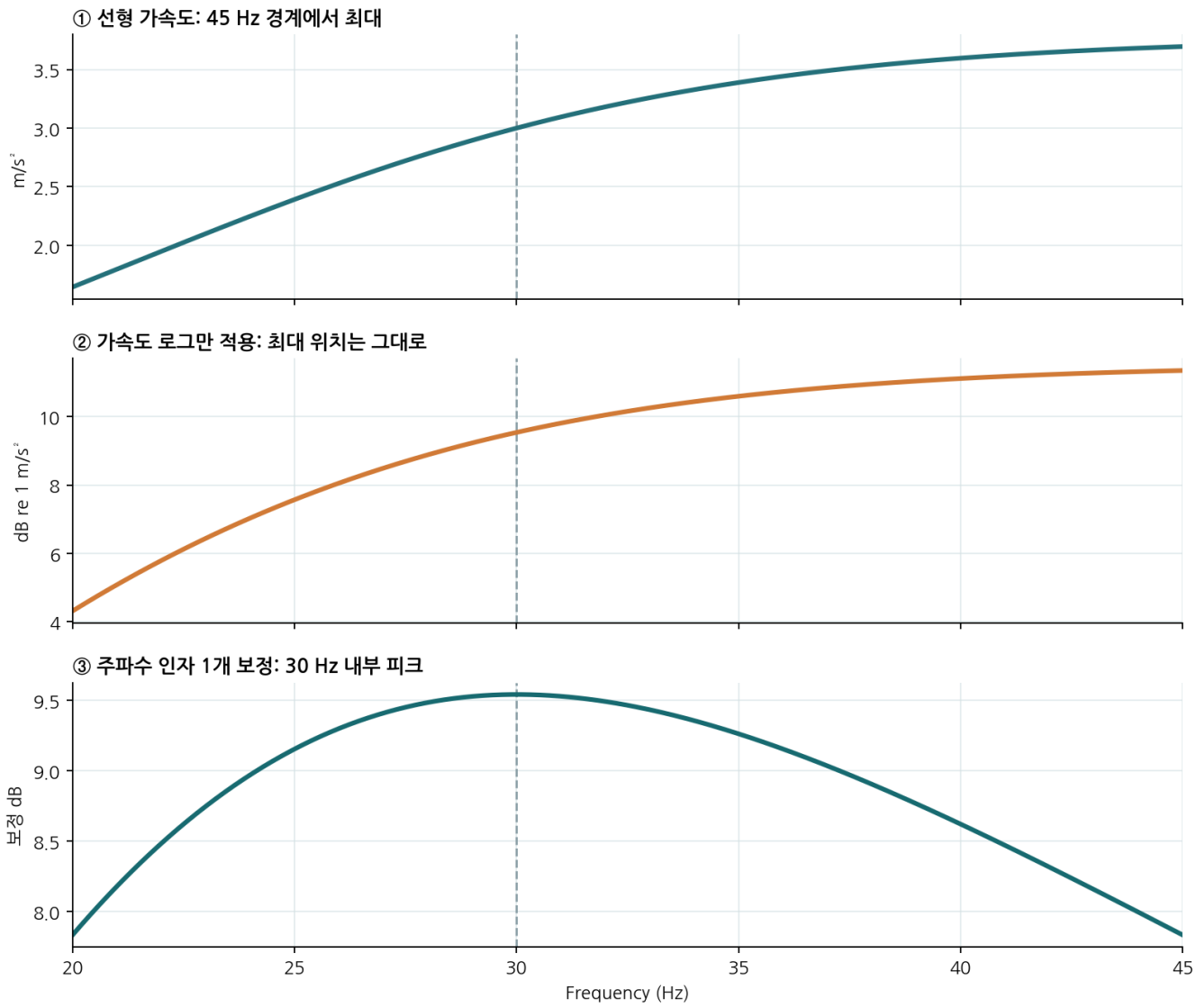
fref = 30 Hz를 기준으로 정하면, 높은 주파수일수록 20 log10(f/30) 만큼 가속도 레벨에서 차감한다. 이는 속도 환산값과 같은 주파수별 순위를 만든다.

주파수	30/f 보정계수	dB 차감량
20 Hz	1.5000	+3.522 dB
25 Hz	1.2000	+1.584 dB
30 Hz	1.0000	0 dB
35 Hz	0.8571	−1.339 dB
40 Hz	0.7500	−2.499 dB
45 Hz	0.6667	−3.522 dB

> 핵심: 30/f는 실험 보정계수가 아니라 수학적 환산계수다. 보정 dB와 속도 dB는 같은 피크 위치를 주며, 두 지표를 독립 검증으로 착각해서는 안 된다.

03. 로그 비교 그래프: 3가지 방식

이론적 MSD 예제 / 실제 흉벽 측정 데이터가 아님



가정: 1자유도, 일정한 정현파 가진력, 무감쇠 고유진동수 30 Hz, 감쇠비 0.6. 이 가상 예에서 가속도와 그 로그는 끝점 최대, 속도로 환산한 보정 dB는 30 Hz의 내부 피크를 보인다.

> 로그 표시만 적용하는 ②는 추천 문제를 해결하지 못한다. ③은 물리적 보정이지만 실제 장치의 가진력 변화까지 보정하지 못한다.

DESIGN NOTE

04. 기존 Arduino 프로젝트에서 달라지는 점

기존 DATA 전송은 유지하고 별도 LOG 출력을 추가했다.

명령	기능
M,30	외부 구동부가 30 Hz로 동작 중일 때 계측
L	측정한 모든 주파수의 선형·로그 지표 출력
S	속도 환산값의 내부 피크를 예비 후보로 판정
C	저장된 주파수 데이터 초기화
X	센서 측정만 중단; 가진기는 별도 정지 필요

기존 웹 파서를 보호하는 출력 형식

DATA 행은 기존 형식(주파수, 가속도 RMS, 변위 RMS, 표본 수, 실효 샘플률, R^2, 품질)을 그대로 유지한다. LOG 행은 사용자가 L 명령을 입력했을 때만 출력한다.

> LOG,Hz,AccelRMS,Accel_dB,VelocityRMS_mmps,Velocity_dB,Corrected_dB,Trials

웹에서 로그로 표시하려면

웹 소스코드가 제공되지 않았으므로 여기서 웹 코드를 직접 수정할 수는 없다. 웹은 기존 DATA의 주파수와 가속도 RMS로 아래 값을 자체 계산하여 Y축을 로그 보조 보기로 표시할 수 있다.

```
const aDb = 20 * Math.log10(accelRms / 1.0);        // dB re 1 m/s²
const vRms = accelRms / (2 * Math.PI * frequencyHz); // m/s
const vDb = 20 * Math.log10(vRms / 0.001);          // dB re 1 mm/s
const corrDb = aDb - 20 * Math.log10(frequencyHz / 30);
```

원래 X축 주파수(Hz)는 그대로 선형으로 둔다. 20~45 Hz처럼 좁은 주파수 범위에서는 X축 로그 변환의 이점이 작다.

DESIGN NOTE

05. 실측값 대입, 판정 기준 및 한계

45 Hz 실제 값 1개만 계산하며 다른 주파수의 측정값은 임의로 만들지 않는다.

45 Hz, A = 6.307 m/s²

값	계산 결과
가속도 레벨	15.996 dB re 1 m/s²
속도 환산 RMS	22.306 mm/s
속도 레벨	26.969 dB re 1 mm/s
30 Hz 기준 보정 레벨	12.475 dB

> 이 숫자는 45 Hz 한 점을 변환한 결과다. 다른 주파수에서 변환된 속도가 더 클 수도 작을 수도 있으므로 공진 후보는 전 구간 데이터로만 판단한다.

추천 기준 및 실패 상태

① 20~45 Hz 스윕의 각 측정 주파수마다 안정화 후 가속도 기본파 RMS를 구한다.
② 속도 RMS 및 보정 dB를 계산한다.
③ 내부 주파수에서 양옆보다 높은 경우만 후보로 보고한다.
④ 20 Hz 또는 45 Hz의 경계 최대는 확정하지 않는다.
⑤ 반복 측정과 입력 가진력 검증 전에는 결론을 “예비 후보”로 제한한다.

아직 해결되지 않은 부분

트랜스듀서 코일 전류와 실제 접촉 전달력은 이 코드에서 측정하지 않는다. 제조사 공칭 공진주파수 또는 임의의 가진력 상수 Kt를 대입해도 개인별 흉벽 공진이 자동으로 보정되지는 않는다. 추가적인 전류·힘 정규화 없이 복합 결합계의 고유진동수를 확정할 수 없다.

$$F_0(f) = K_t I_{coil}(f)$$

위 관계는 코일 전류를 독립적으로 측정할 때 활용할 수 있다. 현재 INA226의 앰프 공급 전류는 코일 교류 전류 RMS가 아니다.

DESIGN NOTE

06. 구현·검증 범위 및 참고자료

코드 부록은 이 문서 안에 전부 수록한다.

하드웨어 / 계측 주의사항

- 트랜스듀서는 이 Arduino 예제에서 제어되지 않는다. M,30 명령 이전에 기존 구동부가 실제로 30 Hz로 작동 중이어야 한다.
- BNO085는 한 축의 부호 있는 가속도를 사용하고, 시각 정보로 기본파 성분을 회귀한다.
- 실제 SDA/SCL, 센서 축, 샘플링률, 과도응답 제외 시간은 현재 실물 환경에 맞게 확인해야 한다.
- 구동기·앰프의 사양과 안전 한계를 넘어 주파수나 출력을 확대하지 않는다.
- 인체 착용 시험은 적절한 연구윤리 및 기계적 안전성 검토 후 실시한다.

검증 상태

보정식의 수학적 동치와 후보 판단 로직을 독립 C++ 테스트로 확인한다. 그러나 현재 실물 장치의 구동 스케치와 결합하여 Arduino IDE에서 빌드하거나, 센서·앰프를 실제 구동하여 검증한 것은 아니다.

자료 출처 / 코드 기준

① 기존 저장 파일: 숨결톡톡_Arduino_공진후보측정_예제.ino (2026-11-02). 측정 루틴을 유지하고 로그 계산·후보 판단을 수정.
② Adafruit BNO08x 라이브러리: sh2_SensorValue의 timestamp 단위는 마이크로초이며, accelerometer 출력을 이용.
③ 기존 MSD 모델링 보고서: 질량-스프링-감쇠 시스템과 구동력·실제 흉벽 접촉 전달력의 차이.

문서 링크: github.com/adafruit/Adafruit_BNO08x / src/sh2_SensorValue.h, src/sh2_SensorValue.c

전체 소스 무결성

별도 첨부된 .ino는 총 388행이며, 아래 부록에 같은 내용을 순서대로 모두 수록했다. SHA-256: `746d4b1593f6b2c0c6c2711b3a95ac2d` …

> 최종 판단: 로그 스케일 도입은 데이터 표현 개선, 속도 환산과 내부 피크 판정은 부분적 알고리즘 개선이다. 둘 모두 입력 힘을 계측한 기계적 FRF를 대신하지 않는다.

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 1/10 · 구동부 미포함

```
/*
 * 숨결톡톡 ¦ Arduino IDE (ESP32 + BNO085) 공진 후보 측정 예제
 * 2026-11-03 / 로그 스케일+속도 보정 통합 측정 코드
 * 중요: 사용자 고유 구동 스케치가 아니며 구동 기능이 없는 측정 전용 예제
 *
 * 목적: 기존 웹 그래프 X=Frequency(Hz), Y=Acceleration(m/s^2) 유지
 *  - 종전의 전체 RMS 대신 구동 주파수 f 성분의 가속도 RMS 추출
 *  - 고주파 끝점 최대를 공진으로 자동 확정하지 않음
 *  - 내부 최대가 발견되면 '예비 후보'만 출력 (입력 가진력 미계측)
 *
 * 중요: 현재 구동 회로/핀/음원 PWM 코드가 제공되지 않았으므로
 *       이 예제는 트랜스듀서를 구동하거나 출력 설정을 변경하지 않는다.
 *       반드시 기존 구동 코드에서 주파수를 먼저 설정하고 안전 확인 후
 *       'M,<실제 구동 주파수>' 명령을 보낼 것.
 *       구동 제어를 통합하려면 해당 장치에 맞는 명령 확인/안정화/정지 로직 필요.
 *
 * 의존성: Arduino IDE, ESP32 core, Adafruit BNO08x 라이브러리
 *  - 측정축은 흉벽에 수직인 축으로 설정 (X/Y/Z 확인 필수)
 *  - 캘리브레이티드 가속도(SH2_ACCELEROMETER) 사용.
 *  - b0(DC)를 회귀로 제거하므로 중력 DC 오프셋은 영향 제한.
 *  - 데이터 유효 대역 및 실제 보고율을 별도 검증.
 *
 * SERIAL: 115200
 * M,30    : 기존 구동 장치를 30 Hz로 안정화한 후 측정 시작
 * RESULTS : DATA,frequencyHz,accelRmsMps2,displacementRmsMm,samples,rateHz,R2,quality
 * S       : 가속도->속도 보정 후 내부 피크 후보 탐색
 * L       : 저장된 각 주파수의 로그 스케일 및 속도 지표 출력
 * C       : 저장한 결과 초기화
 * X       : 측정 취소(트랜스듀서 정지는 기존 구동 코드에서 별도 처리!)
 */

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_BNO08x.h>
#include <math.h>
#include <stdlib.h>
#include <string.h>

static constexpr uint32_t SERIAL_BAUD = 115200;
static constexpr uint32_t SENSOR_INTERVAL_US = 4000; // 목표 250Hz, 실제값은 계측 확인
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 2/10 · 구동부 미포함

```
041  static constexpr uint32_t SETTLE_MS = 800; // 실험 조건에 따라 검증 필요
042  static constexpr uint32_t MEASURE_MS = 1600;
043  static constexpr size_t MAX_SAMPLES = 800;
044  static constexpr size_t MAX_POINTS = 48;
045  static constexpr float MIN_F_HZ = 20.0f; // 현재 검증된 탐색 범위로 제한
046  static constexpr float MAX_F_HZ = 45.0f;
047  static constexpr char MEASURE_AXIS = 'Z'; // 반드시 실제 흉벽 법선 축 확인
048  static constexpr double PI_D = 3.14159265358979323846;
049  // 로그 레벨은 반드시 참조량을 함께 기록한다.
050  static constexpr double ACC_REF_MPS2 = 1.0;    // 0 dB re 1 m/s^2
051  static constexpr double VEL_REF_MPS = 0.001;   // 0 dB re 1 mm/s
052  static constexpr double F_REF_HZ = 30.0;       // 주파수 환산 비교 기준
053
054  // 아래 계산은 단일 정현파의 가속도 RMS에만 적용한다.
055  double velocityRms(double aRms, double fHz) {
056    if (!isfinite(aRms) || !isfinite(fHz) || aRms <= 0 || fHz <= 0) return NAN;
057    return aRms / (2.0 * PI_D * fHz);
058  }
059
060  double accelDb(double aRms) {
061    if (!isfinite(aRms) || aRms <= 0) return NAN;
062    return 20.0 * log10(aRms / ACC_REF_MPS2);
063  }
064
065  double velocityDb(double aRms, double fHz) {
066    double v = velocityRms(aRms, fHz);
067    if (!isfinite(v) || v <= 0) return NAN;
068    return 20.0 * log10(v / VEL_REF_MPS);
069  }
070
071  // 가속도를 기준 주파수 F_REF_HZ로 환산한 보정 로그 레벨.
072  // v = a/(2*pi*f)에 고정 상수를 곱한 것이므로 피크 위치는 velocityDb와 같다.
073  double correctedDb(double aRms, double fHz) {
074    if (!isfinite(fHz) || fHz <= 0) return NAN;
075    double aLevel = accelDb(aRms);
076    if (!isfinite(aLevel)) return NAN;
077    return aLevel - 20.0 * log10(fHz / F_REF_HZ);
078  }
079
080
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 3/10 · 구동부 미포함

```
Adafruit_BNO08x bno08x(-1);
sh2_SensorValue_t sensorValue;

enum Phase { IDLE, SETTLING, CAPTURING };
Phase phase = IDLE;
uint32_t phaseStartMs = 0;
float targetHz = 0.0f;

struct Sample { uint64_t timeUs; float a; };
Sample samples[MAX_SAMPLES];
size_t sampleCount = 0;
uint64_t lastTimeUs = 0;
uint32_t nonmonotonicTime = 0;
uint32_t droppedEvents = 0;
uint8_t prevSequence = 0;
bool hasPrevSequence = false;

struct Point {
  float hz;
  float meanRms;
  float sumRms;
  float sumRmsSq;
  uint8_t trials;
};
Point points[MAX_POINTS];
size_t pointCount = 0;

bool getAxisAcceleration(const sh2_SensorValue_t &ev, float &out) {
  if (ev.sensorId != SH2_ACCELEROMETER) return false;
  switch (MEASURE_AXIS) {
    case 'X': out = ev.un.accelerometer.x; break;
    case 'Y': out = ev.un.accelerometer.y; break;
    case 'Z': out = ev.un.accelerometer.z; break;
    default: return false;
  }
  return isfinite(out);
}

// 3x3 연립방정식을 부분 피벗팅 가우스 소거로 풀이.
// x(t)=b0+bC*cos(2*pi*f*t)+bS*sin(2*pi*f*t)
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 4/10 · 구동부 미포함

```
bool solve3x3(double aug[3][4], double x[3]) {
  for (int k = 0; k < 3; ++k) {
    int best = k;
    for (int r = k + 1; r < 3; ++r)
      if (fabs(aug[r][k]) > fabs(aug[best][k])) best = r;
    if (fabs(aug[best][k]) < 1.e-9) return false;
    for (int j = k; j < 4; ++j) {
      double v = aug[k][j]; aug[k][j] = aug[best][j]; aug[best][j] = v;
    }
    double d = aug[k][k];
    for (int j = k; j < 4; ++j) aug[k][j] /= d;
    for (int r = 0; r < 3; ++r) if (r != k) {
      double q = aug[r][k];
      for (int j = k; j < 4; ++j) aug[r][j] -= q * aug[k][j];
    }
  }
  for (int k = 0; k < 3; ++k) x[k] = aug[k][3];
  return true;
}

struct FitResult {
  bool ok;
  double rms;
  double r2;
  double sampleRate;
  double displacementMm;
};

FitResult fitFrequency(float f) {
  FitResult result = {false, NAN, NAN, NAN, NAN};
  if (sampleCount < 64 || !(f > 0)) return result;
  uint64_t t0 = samples[0].timeUs;
  uint64_t t1 = samples[sampleCount-1].timeUs;
  if (t1 <= t0) return result;
  double dt = ((double)(t1 - t0)) * 1.e-6;
  if (dt < 1.0) return result;
  result.sampleRate = (sampleCount - 1) / dt;
  double sum[3][3] = {{0}}, rhs[3] = {0};
  for (size_t i = 0; i < sampleCount; ++i) {
    double t = ((double)(samples[i].timeUs - t0))*1.e-6;
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 5/10 · 구동부 미포함

```
    double ang = 2.0 * PI_D * (double)f * t;
    double x[3] = {1.0, cos(ang), sin(ang)};
    for (int r=0; r<3; ++r) {
      rhs[r] += x[r] * samples[i].a;
      for (int c=0; c<3; ++c) sum[r][c] += x[r]*x[c];
    }
  }
  double aug[3][4];
  for (int r=0; r<3; ++r) {
    for (int c=0; c<3; ++c) aug[r][c] = sum[r][c];
    aug[r][3] = rhs[r];
  }
  double b[3];
  if (!solve3x3(aug,b)) return result;
  double mean = 0;
  for (size_t i=0;i<sampleCount;i++) mean += samples[i].a;
  mean /= sampleCount;
  double sse=0,sst=0;
  for (size_t i=0;i<sampleCount;i++) {
    double t=((double)(samples[i].timeUs-t0))*1.e-6;
    double ang=2.0*PI_D*(double)f*t;
    double fitted=b[0]+b[1]*cos(ang)+b[2]*sin(ang);
    double e=(double)samples[i].a-fitted;
    sse+=e*e;
    double centered=(double)samples[i].a-mean;
    sst+=centered*centered;
  }
  result.r2 = sst > 1.e-12 ? 1.0 - sse/sst : NAN;
  double peak=sqrt(b[1]*b[1]+b[2]*b[2]);
  result.rms=peak/sqrt(2.0);
  result.displacementMm=1000.0*result.rms/pow(2.0*PI_D*f,2);
  result.ok=isfinite(result.rms) && isfinite(result.r2);
  return result;
}

void addPoint(float f, float rms) {
  size_t at=pointCount;
  for (size_t i=0;i<pointCount;++i)
    if (fabs(points[i].hz-f)<0.01f) {at=i;break;}
  if (at==pointCount) {
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 6/10 · 구동부 미포함

```
    if (pointCount>=MAX_POINTS) { Serial.println("ERR,POINT_TABLE_FULL"); return; }
    points[at]={f,0,0,0,0}; pointCount++;
  }
  Point &p=points[at];
  if (p.trials==255) { Serial.println("ERR,TRIAL_LIMIT");return; }
  p.sumRms+=rms; p.sumRmsSq+=rms*rms; p.trials++;
  p.meanRms=p.sumRms/p.trials;
}

void reportFit() {
  FitResult r=fitFrequency(targetHz);
  bool rateOK = r.ok && r.sampleRate >= 5.0*targetHz;
  bool timeOK = (nonmonotonicTime==0);
  bool fitOK = r.ok && r.r2>=0.60; // 연구용 예시, 보편적 인증 기준 아님
  bool valid = rateOK && timeOK && fitOK && droppedEvents==0;
  const char *quality = !r.ok ? "INVALID_FIT" :
     !timeOK ? "BAD_TIMESTAMP" :
     !rateOK ? "LOW_SAMPLE_RATE" :
     droppedEvents ? "DROPPED_SAMPLES" :
     !fitOK ? "LOW_FIT_R2" : "MEASURE_OK";
  Serial.print("DATA,"); Serial.print(targetHz,3);
  Serial.print(','); Serial.print(r.rms,6);
  Serial.print(','); Serial.print(r.displacementMm,6);
  Serial.print(','); Serial.print(sampleCount);
  Serial.print(','); Serial.print(r.sampleRate,2);
  Serial.print(','); Serial.print(r.r2,3);
  Serial.print(','); Serial.println(quality);
  // 결과가 유효할 때만 '예비 후보' 판정에 포함
  if (valid) addPoint(targetHz,(float)r.rms);
  phase=IDLE;
  Serial.println("STATE,IDLE");
}

// 로그 스케일 출력은 별도 명령(L)에서만 수행해 기존 DATA 포맷을 깨지 않는다.
void printLogTable() {
  if(pointCount==0) { Serial.println("LOG,NO_POINTS"); return; }
  Point arr[MAX_POINTS];
  memcpy(arr, points, sizeof(Point)*pointCount);
  for(size_t i=0;i<pointCount;i++)
    for(size_t j=i+1;j<pointCount;j++)
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 7/10 · 구동부 미포함

```
      if(arr[j].hz<arr[i].hz) { Point t=arr[i]; arr[i]=arr[j]; arr[j]=t; }
  Serial.println("LOG_HEADER,Hz,AccelRMS_mps2,Accel_dB_re_1mps2,VelocityRMS_mmps,Velocity_dB_re_1mmps,Corrected_dB_re_30Hz,Trials");
  for(size_t i=0;i<pointCount;i++) {
    const Point &p=arr[i];
    double v=velocityRms(p.meanRms,p.hz);
    Serial.print("LOG,"); Serial.print(p.hz,3);
    Serial.print(','); Serial.print(p.meanRms,6);
    Serial.print(','); Serial.print(accelDb(p.meanRms),3);
    Serial.print(','); Serial.print(v*1000.0,6);
    Serial.print(','); Serial.print(velocityDb(p.meanRms,p.hz),3);
    Serial.print(','); Serial.print(correctedDb(p.meanRms,p.hz),3);
    Serial.print(','); Serial.println(p.trials);
  }
}

void printCandidates() {
  if (pointCount<3) {Serial.println("SWEEP,INSUFFICIENT_POINTS"); return;}
  Point arr[MAX_POINTS];
  memcpy(arr,points,sizeof(Point)*pointCount);
  for (size_t i=0;i<pointCount;i++)
    for (size_t j=i+1;j<pointCount;j++) if(arr[j].hz<arr[i].hz) {
      Point tmp=arr[i];arr[i]=arr[j];arr[j]=tmp;
    }
  // 기존과 달리 가속도 절대 최대가 아니라 속도 환산값으로 판정한다.
  int best=0;
  for (size_t i=1;i<pointCount;i++)
    if(velocityRms(arr[i].meanRms,arr[i].hz) >
       velocityRms(arr[best].meanRms,arr[best].hz)) best=(int)i;
  if(best==0 || best==(int)pointCount-1)
    Serial.println("SWEEP,BOUNDARY_MAX_NOT_RESONANCE");
  int count=0;
  for(size_t i=1;i+1<pointCount;i++) {
    Point &p=arr[i];
    double vHere=velocityRms(p.meanRms,p.hz);
    double vLeft=velocityRms(arr[i-1].meanRms,arr[i-1].hz);
    double vRight=velocityRms(arr[i+1].meanRms,arr[i+1].hz);
    if(!(isfinite(vHere) && isfinite(vLeft) && isfinite(vRight)) ||
       vHere<=vLeft || vHere<=vRight) continue;
    double cv=INFINITY;
    if(p.trials>=3 && p.meanRms>1e-9) {
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 8/10 · 구동부 미포함

```
      double var=((double)p.sumRmsSq/p.trials-p.meanRms*p.meanRms);
      cv=sqrt(fmax(0.0,var))/p.meanRms;
    }
    Serial.print("CANDIDATE,");Serial.print(p.hz,3);
    Serial.print(',');Serial.print(p.meanRms,6);
    Serial.print(',');Serial.print(p.trials);
    Serial.print(',');Serial.print(cv,3);
    Serial.print(',');Serial.println(cv <= 0.15 ? "REPEATABLE_PRELIMINARY" : "NEEDS_REPEAT");
    Serial.print("V_SCORE,"); Serial.print(p.hz,3);
    Serial.print(',');Serial.print(vHere*1000.0,5);  // mm/s
    Serial.print(',');Serial.println(correctedDb(p.meanRms,p.hz),3);
    count++;
  }
  if(count==0) Serial.println("SWEEP,NO_INTERIOR_PEAK");
  // '공진 확정' 문구를 절대 출력하지 않는다.
}

void startMeasure(float hz) {
  if(phase!=IDLE) { Serial.println("ERR,BUSY");return; }
  if(!isfinite(hz) || hz<MIN_F_HZ || hz>MAX_F_HZ) {
    Serial.println("ERR,FREQUENCY_OUT_OF_VALIDATED_RANGE");return;
  }
  targetHz=hz;
  sampleCount=0; lastTimeUs=0; nonmonotonicTime=0;
  droppedEvents=0; hasPrevSequence=false;
  phase=SETTLING; phaseStartMs=millis();
  Serial.print("STATE,SETTLING,");Serial.println(targetHz,3);
  // 트랜스듀서는 이 펌웨어가 제어하지 않음. 호스트/기존 펌웨어에서 설정 필수!
}

char commandBuf[48]={0};
size_t commandLen=0;
void dispatchCommand(char *line) {
  if(strcmp(line,"S")==0){printCandidates();return;}
  if(strcmp(line,"L")==0){if(phase==IDLE) printLogTable();
    else Serial.println("ERR,BUSY_LOG_TABLE");return;}
  if(strcmp(line,"C")==0){if(phase==IDLE){pointCount=0;Serial.println("STATE,CLEARED");}return;}
  if(strcmp(line,"X")==0){phase=IDLE;Serial.println("STATE,MEASUREMENT_ABORTED_ONLY");return;}
  if(line[0]=='M' && line[1]==','){
    char *endp=nullptr;
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 9/10 · 구동부 미포함

```
    float hz=strtof(line+2,&endp);
    if(endp==line+2 || *endp!='\0') {Serial.println("ERR,BAD_FREQUENCY");return;}
    startMeasure(hz);return;
  }
  Serial.println("ERR,UNKNOWN_COMMAND");
}

void handleSerial() {
  while(Serial.available()) {
    char c=(char)Serial.read();
    if(c=='\r') continue;
    if(c=='\n') {commandBuf[commandLen]='\0';dispatchCommand(commandBuf);commandLen=0;}
    else if(commandLen+1<sizeof(commandBuf)) commandBuf[commandLen++]=c;
    else {commandLen=0;Serial.println("ERR,COMMAND_TOO_LONG");}
  }
}

void setup() {
  Serial.begin(SERIAL_BAUD);
  Wire.begin(); // 사용 중인 ESP32의 실제 SDA/SCL 핀 설정 유지/확인
  Wire.setClock(400000); // 기판/배선 조건에 따라 변경하고 실제 샘플링 확인
  if (!bno08x.begin_I2C()) {
    Serial.println("ERR,BNO085_NOT_FOUND");
    while(1) {delay(1000);}
  }
  if (!bno08x.enableReport(SH2_ACCELEROMETER,SENSOR_INTERVAL_US)) {
    Serial.println("ERR,CANNOT_ENABLE_ACCELEROMETER");
    while(1) {delay(1000);}
  }
  Serial.println("READY,NO_ACTUATOR_CONTROL,M-S-L-C-X-COMMANDS");
}

void loop() {
  handleSerial();
  if(bno08x.wasReset()) {
    phase=IDLE;pointCount=0;
    if(!bno08x.enableReport(SH2_ACCELEROMETER,SENSOR_INTERVAL_US))
      Serial.println("ERR,BNO085_RESET_REPORT_FAILURE");
    else Serial.println("WARN,BNO085_RESET_SWEEP_CLEARED");
  }
```

부록 | 전체 Arduino IDE 측정 코드

388행 원본 순서대로 수록 · 코드 페이지 10/10 · 구동부 미포함

```
361
362   // 측정 중 데이터가 뒤늦게 들어오는 것을 방지하기 위해 정착 단계에서도 읽어 버림.
363   for(int i=0;i<12 && bno08x.getSensorEvent(&sensorValue);++i) {
364     float a=0;
365     if(!getAxisAcceleration(sensorValue,a)) continue;
366     if(phase!=CAPTURING) continue;
367     if(sampleCount>=MAX_SAMPLES) continue;
368     uint64_t t=sensorValue.timestamp;
369     if(t==0 || (sampleCount>0 && t<=lastTimeUs)) {
370       nonmonotonicTime++;continue;
371     }
372     if(hasPrevSequence) {
373       uint8_t diff=(uint8_t)(sensorValue.sequence-prevSequence);
374       if(diff!=1) droppedEvents+=(diff>0?diff-1:1);
375     }
376     prevSequence=sensorValue.sequence;hasPrevSequence=true;
377     samples[sampleCount++]={t,a};lastTimeUs=t;
378   }
379
380   uint32_t now=millis();
381   if(phase==SETTLING && (uint32_t)(now-phaseStartMs)>=SETTLE_MS) {
382     phase=CAPTURING;phaseStartMs=now;
383     sampleCount=0;lastTimeUs=0;nonmonotonicTime=0;
384     droppedEvents=0;hasPrevSequence=false;
385     Serial.println("STATE,CAPTURING");
386   }
387   if(phase==CAPTURING && (uint32_t)(now-phaseStartMs)>=MEASURE_MS) reportFit();
388 }
```